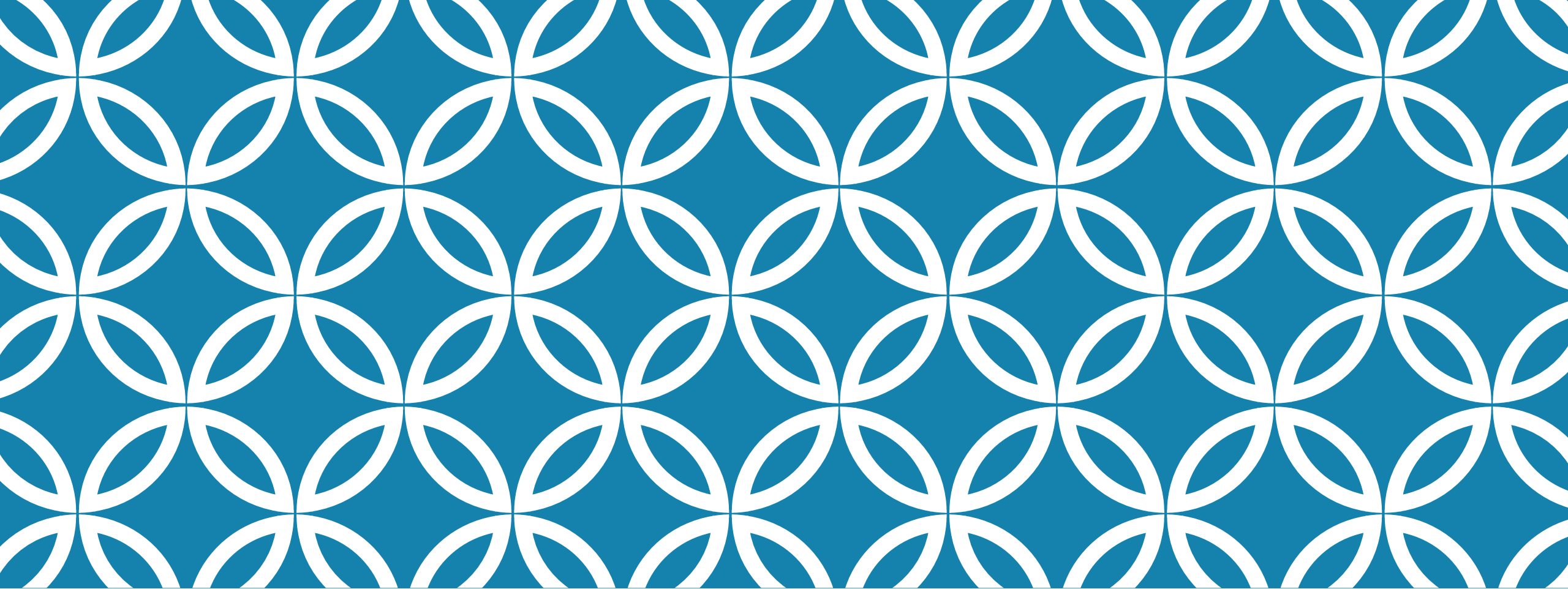




STATIC PROGRAM ANALYSIS

Lecture 3 – Abstract Syntax Tree

ABSTRACT SYNTAX TREE

- Program representation with an added semantics over just text
- Ordered tree over program elements
 - leaves: variables, literals
 - internal nodes: block, statements, expressions
 - a derivation from the language's grammar
- Commonly traversed using visitors pattern

AST PARSERS FOR JAVA (FRONT-ENDS)

- Polyglot (<https://www.cs.cornell.edu/projects/polyglot/>)
- Javaparser (<https://javaparser.org/>)
- Eclipse's org.eclipse.jdt.core.dom package – can only use inside Eclipse  Used in Wala static analyzer
- Spoon (<https://github.com/INRIA/spoon>) allows to compile outside of Eclipse, still using Eclipse's Java Development Tools (JDT)

TRAVERSING AST

- Using visitors pattern: separates data-structure from functionality
 - A tree with a root node
 - Nodes are of different types, but implement the same interface that accepts a visitor of some NodeVisitor interface type
 - `public void accept(NodeVisitor visitor){ visitor.visit(this);}`
 - NodeVisitor interface/abstract class has visit methods for each type of node, e.g., method, conditional etc.
 - `visit(: Method)`
 - `visit(: IfStmt)`
 - A concrete visitor NodeCounterVisitor implements NodeVisitor (and thus instantiate its methods or overrides needed methods)
 - Instructs exactly how to process each visit(Type argNode)
 - Count or skip
 - Decent into the node's children or not
 - If a visitor returns void – get children and invoke accept on them, i.e., `rhs = argNode.getRhs(), rhs.accept(this)`
 - If a visitor returns a Boolean – return true – visit children, return false – don't visit children
-  This is what Eclipse's visitor does

ECLIPSE VISITOR

- A concrete visitor extends abstract ASTVisitor class
- Override methods that needs to be modified
 - Each by default return true (visit all children)
- Each can has 4 overridden methods
 - void preVisit(NodeType node){ }
 - boolean visit(NodeType node){return true}
 - void endVisit(NodeType node){ }
 - void postVisit(NodeType node){ }
- Definitely override with return false those that you don't need to traverse (e.g., Initializer)
- Override pre/end/postVisit if you need to set up something before the node, after you are done visiting its children and returning from visit(), and after completing endVisit.

```
private class AnalyzerVisitor extends ASTVisitor {  
  
    @Override  
    public boolean visit(Initializer node) {  
        return false;  
    }  
  
    @Override  
    public boolean visit(NormalAnnotation node) {  
        return false;  
    }  
  
    @Override  
    public boolean visit(MarkerAnnotation node) {  
        return false;  
    }  
}
```

ECLIPSE VISITOR (CONT)

- Check documentation for all possible node types
<https://git.eclipse.org/c/jdt/eclipse.jdt.core.git/tree/org.eclipse.jdt.core/dom/org/eclipse/jdt/core/dom/ASTVisitor.java>
- Eclipse also uses Java Model representation of a project.
 - Light-weight
 - Contains some info as AST, but fast
 - Outline view uses the Java model

SOLVE THE SAME PROBLEMS BUT WITH AST

- A. Define N threshold to quantify “large”; compute the number of lines M in a code; if $M > N$ then the answer is yes.
- B. Define B threshold to quantify “too many”; compute the number C of if statements; if $C > B$ then the answer is yes.
- C. Does a program have if/else/then branch, or a nested conditional statement?

IN-CLASS WORK

Get the start code

<https://github.com/BoiseState/static-program-analysis>

How to start parsing a Java file

```
public void analyze() throws IOException {
    String source = new String(Files.readAllBytes(file.toPath()));
    ASTParser parser = ASTParser.newParser(AST.JLS3);
    parser.setSource(source.toCharArray());
    parser.setKind(ASTParser.K_COMPILATION_UNIT);
    ASTNode node = parser.createAST(null);
    ProcessorVisitor visitor = new ProcessorVisitor();
    node.accept(visitor);
}
```

Your custom visitor, perhaps with some instance variable

```
private class ProcessorVisitor extends ASTVisitor{
    @Override
    public boolean visit(TypeDeclaration node) {
        System.out.println("Here! " + node.getName());
        return true;
    }

    @Override
    public boolean visit(MethodDeclaration node) {
        System.out.println("In method " + node.getName());
        return false;
    }
}
```




NEXT TIME

- New problems
- AST manipulation (?)
- Flow graphs